Hindi Singing ASR

Fine-tuned Whisper for Hindi song lyrics transcription.

Saregama × IIT Bombay

Setup

Install the required dependencies: **pip install -r requirements.txt**

All scripts are run from the repository root: **hindi-song-asr/**

Scripts

1. Vocal Extraction

- **Script**
  - `data_prep/dataset/vocals.py`
- **Purpose**
  - Takes raw MP3 song files and extracts isolated vocal tracks using Demucs.
- **Run**
  - python data_prep/dataset/vocals.py
- **Needs**
  - MP3 files in `data/downloads/audio/`
  - `demucs` package installed
- **Produces**
  - WAV vocal tracks in `data/downloads/vocals/`

2. Training

- **Script**
  - `training_scripts/train_whisper_medium_encdec_lora.py`
- **Purpose**
  - LoRA fine-tuning of Whisper Large-v3 Turbo on encoder + decoder attention layers. Uses DDP for multi-GPU training.

- **Run**
  - **torchrun --nproc_per_node=2 training_scripts/train_whisper_medium_encdec_lora.py**
- **Needs**
  - Manifest JSONL files in:
    - `data/training_data_final/manifests/`
      - `train.jsonl`
      - `val.jsonl`
      - `test.jsonl`

1. Each manifest line has: {"audio": "path/to/chunk.wav", "text": "transcript"}
2. The audio paths inside the manifests must point to actual WAV chunk files in:
   a. `data/training_data_final/chunks/`
3. WandB account (logs to project **`whisper-singing-asr`**)

Produces

- LoRA adapter checkpoints:
  - `outputs/whisper-large-v3-turbo-lora/`
- Training metrics logged to WandB

Configuration (hardcoded at top of file)

- `MODEL_ID` = `openai/whisper-large-v3-turbo`
- `LANGUAGE` = `hi`
- LoRA rank = 32
- LoRA alpha = 64
- Dropout = 0.05
- 20 epochs
- Batch size = 32
- Learning rate = 1e-4
- Evaluation every 500 steps
- Saves best 3 checkpoints by CER

---

3. Inference (Greedy / Beam)

- **Script**
  - `inference/infer_generic.py`
- **Purpose**
  - Runs inference on the test split using a trained LoRA adapter.
  - Supports:
    - Greedy decoding
    - Beam search
    - Beam search with repetition penalty

Run

- **Greedy (default)**
  - python inference/infer_generic.py
- **Beam search**
  - python inference/infer_generic.py --decode_strategy beam
- **Different adapter**
  - python inference/infer_generic.py --model_id openai/whisper-large-v3-turbo --adapter outputs/whisper-large-v3-turbo-lora --language hi

Needs

- Test manifest:
  - `data/training_data_final/manifests/test.jsonl`
- Audio WAV files referenced in the manifest
- Trained LoRA adapter directory

Default:

- `outputs/whisper-medium-enc-lora`

Produces

- `predictions/<model>_<strategy>/test_predictions_<timestamp>.jsonl` (Per-sample CER/WER)
- `predictions/<model>_<strategy>/test_predictions_<timestamp>.txt` (Human-readable output)
- `predictions/<model>_<strategy>/test_summary_<timestamp>.json` (Corpus CER/WER and timing)

Arguments

Argument	Default	Description
`--model_id`	**`openai/whisper-medium`**	Base Whisper model
`--adapter`	**`outputs/whisper-medium-enc-lora`**	Path to LoRA adapter
`--language`	**`hi`**	Language code
`--manifest`	**`data/training_data_final/manifests/test.jsonl`**	Test manifest
`--decode_strategy`	**`greedy`**	`greedy`, `beam`, or `beam_reppen`
`--batch_size`	**`16`**	Batch size
`--limit`	**`0`**	Max samples (`0 = all`)

4. Inference (N-best Beam)

- **Script**
  - **`inference/infer_beam.py`**
- **Purpose**
  - Runs N-best beam search inference.
  - Returns multiple hypotheses per sample together with log-probabilities (useful for rescoring).
- **Run**
  - Default
  - **python inference/infer_beam.py**
- **Custom settings**
  - **python inference/infer_beam.py --model_id openai/whisper-large-v3-turbo --adapter outputs/whisper-large-v3-turbo-lora --num_beams 10 --num_return_sequences 5**
- **Needs**
  - • Test manifest:
    - `data/training_data_final/manifests/test.jsonl`
    - • Audio WAV files referenced in the manifest
    - • Trained LoRA adapter directory
- **Default**:
  - `outputs/whisper-large-v3-turbo-lora`
- **Produces**
  - `predictions/<model>_nbest_b<beams>/nbest_<timestamp>.jsonl`(N hypotheses per sample with scores)
  - `predictions/<model>_nbest_b<beams>/nbest_summary_<timestamp>.json`

(Top-1 corpus CER/WER)

- **Arguments**

Argument	Default	Description
`--model_id`	`openai/whisper-large-v3-turbo`	Base Whisper model
`--adapter`	`outputs/whisper-large-v3-turbo-lora`	Path to LoRA adapter
`--language`	`hi`	Language code
`--manifest`	`data/training_data_final/manifests/test.jsonl`	Test manifest
`--num_beams`	`5`	Beam width
`--num_return_sequences`	`5`	Hypotheses per sample (must be ≤ `num_beams`)
`--batch_size`	`32`	Batch size
`--limit`	`0`	Max samples (`0 = all`)
`--save_token_ids`	`off`	Include token IDs in output

Data Layout

data/training_data_final/

- **manifests/**
  - `train.jsonl`
  - `val.jsonl`
  - `test.jsonl`
- **chunks/**
  - **INH100017020_E/**
    - `00001.wav`
    - `00002.wav`
    - `...`
  - `...`
- `clean_dataset.json`
- `split.json`

Where:

- `train.jsonl`, `val.jsonl`, `test.jsonl` contain entries of the form:

{"audio": "data/training_data_final/chunks/SONG_ID/00001.wav", "text": "..."}

- `clean_dataset.json` contains the source lyrics with timestamps.
- `split.json` specifies which songs belong to the train, validation, and test splits.

The manifests are the entry point for training and inference.

Each line is a JSON object containing:

- an `audio` path
- a `text` transcript

The audio paths must resolve to real WAV files.